

LN 4. 感知机

张晓今

计算机学院数据挖掘与机器学习实验室

华中科技大学 Hopcroft 计算科学研究中心

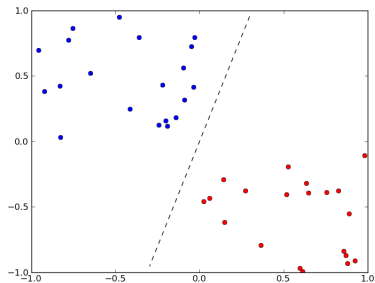


- 1 概念
 - 假设
- 2 分类器
 - 参数选择
 - 超平面
- 3 感知机算法
 - 感知机算法
 - 几何直觉
- 4 感知机的收敛性
 - 感知机的收敛性
 - 定理与证明
- 5 感知机的历史
 - 感知机的历史
 - 从感知机到人工神经元
- 6 几何直觉

- 1 概念
 - 假设
- 2 分类器
 - 参数选择
 - 超平面
- 3 感知机算法
 - 感知机算法
 - 几何直觉
- 4 感知机的收敛性
 - 感知机的收敛性
 - 定理与证明
- 5 感知机的历史
 - 感知机的历史
 - 从感知机到人工神经元
- 6 几何直觉

假设

- 二分类 (即 $y_i \in \{-1, +1\}$)
- 数据线性可分



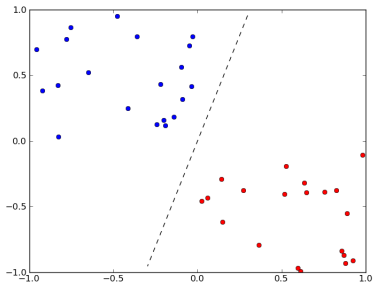
假设

基本思想:

- 在机器学习中，感知器是一种用于监督学习的二分类器。
- 二分类器是一个函数，决定由数字向量表示的输入是否属于某个特定的类。
- 它是一种线性分类器，即基于一组权重与特征向量相结合的线性预测函数来进行预测的分类算法。

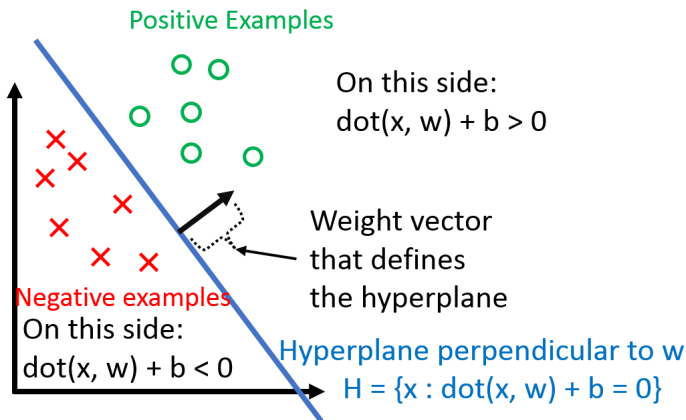
$$\text{假设空间: } \mathcal{H} = \{h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = 0\}$$

对应于特征空间中的一个超平面，其中： $\mathbf{w}$ 是超平面的法向量， b 是超平面的截距。



- 1 概念
 - 假设
- 2 分类器
 - 参数选择
 - 超平面
- 3 感知机算法
 - 感知机算法
 - 几何直觉
- 4 感知机的收敛性
 - 感知机的收敛性
 - 定理与证明
- 5 感知机的历史
 - 感知机的历史
 - 从感知机到人工神经元
- 6 几何直觉

$$h(x_i) = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b)$$



b 是偏置项 (如果没有偏置项, $\mathbf{w}$ 定义的超平面将始终经过原点)。

处理 b 可能很麻烦, 所以通过添加一个额外的常量维度将它“吸收”到特征向量 $\mathbf{w}$ 中。
在该约定下:

$$\begin{aligned} \mathbf{x}_i & \text{ 变为 } \begin{bmatrix} \mathbf{x}_i \\ 1 \end{bmatrix} \\ \mathbf{w} & \text{ 变为 } \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} \end{aligned}$$

可以验证:

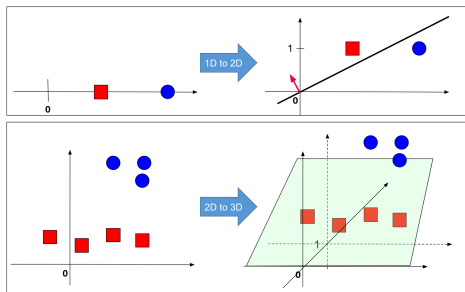
$$\begin{bmatrix} \mathbf{x}_i \\ 1 \end{bmatrix}^\top \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix} = \mathbf{w}^\top \mathbf{x}_i + b$$

从而得到:

$$\mathcal{H} = \{h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = 0\}$$

然后，我们可以将上述的 $h(\mathbf{x}_i)$ 简化为

$$h(\mathbf{x}_i) = \text{sign}(\mathbf{w}^\top \mathbf{x}_i)$$



(左:) 原始数据是一维 (上图) 还是二维 (下图)。不存在一个穿过原点的超平面可将红点和蓝点分开。

(右:) 对所有数据点加一个常量维后，这样的超平面就存在了。

观察

请注意,

$$y_i(\mathbf{w}^\top \mathbf{x}_i) > 0 \iff \mathbf{x}_i \text{ 分类正确}$$

其中“分类正确”意味着 $\mathbf{x}_i$ 在由 $\mathbf{w}$ 定义的超平面的正确一侧。
另外, 左边依赖于 $y_i \in \{-1, +1\}$ (若 $y_i \in \{0, +1\}$, 就不起作用了)。

1 概念

- 假设

2 分类器

- 参数选择
- 超平面

3 感知机算法

- 感知机算法
- 几何直觉

4 感知机的收敛性

- 感知机的收敛性
- 定理与证明

5 感知机的历史

- 感知机的历史
- 从感知机到人工神经元

6 几何直觉

我们知道了 w 应该做什么 (定义一个分离数据的超平面),
接下来看看如何获得这样的 w 。

```

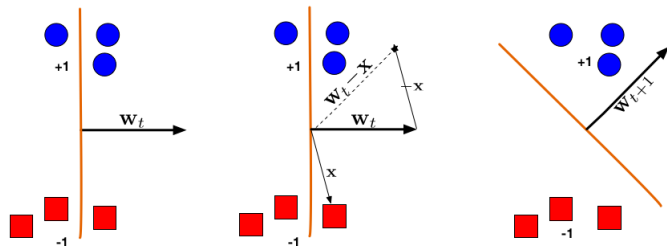
Initialize  $\vec{w} = \vec{0}$ 
while TRUE do
   $m = 0$ 
  for  $(x_i, y_i) \in D$  do
    if  $y_i(\vec{w}^T \cdot \vec{x}_i) \leq 0$  then
       $\vec{w} \leftarrow \vec{w} + y_i \vec{x}$ 
       $m \leftarrow m + 1$ 
    end if
  end for
  if  $m = 0$  then
    break
  end if
end while

// Initialize  $\vec{w}$ .  $\vec{w} = \vec{0}$  misclassifies everything.
// Keep looping
// Count the number of misclassifications,  $m$ 
// Loop over each (data, label) pair in the dataset,  $D$ 
// If the pair  $(\vec{x}_i, y_i)$  is misclassified
// Update the weight vector  $\vec{w}$ 
// Counter the number of misclassification

// If the most recent  $\vec{w}$  gave 0 misclassifications
// Break out of the while-loop

// Otherwise, keep looping!

```



感知器更新的示例：

(左:) 由 w_t 定义的超平面错误地分类了一个红点 (-1) 和一个蓝点 ($+1$)。

(中:) 红点 x 被选中并用于更新。因为它的标签是 -1 ，我们需要从 w_t 中减去 x 。

(右:) 已更新的超平面 $w_{t+1} = w_t - x$ 正确分离了两个类，感知机算法已经收敛。

前提:

所在数据点线性可分，即可以找到一个超平面将数据点正确分开。

算法的直观解释:

对所有数据点进行枚举:

当发现一个数据点被当前超平面分类错误，则进行调整： $\mathbf{w} = \mathbf{w} + y_i \mathbf{x}_i$

使分类超平面向该误分类点的一侧移动，以减小该误分类点与超平面间的距离，直至所有数据点正确分类。

注意:

对分类点的枚举顺序不同，对应的误分类点的顺序不同，可能会得到不同的分类超平面。

测试

1. 假设数据集仅由单个数据点 $\{(x, +1)\}$ 组成。感知器对这个点 x 反复错误分类的频率是多少？
2. 如果初始权重向量 w 是随机初始化的，而不是全零向量会怎样？
3. 对如下训练数据：

$$x_1 = (3, 3)^T, y_1 = 1,$$

$$x_2 = (4, 2)^T, y_2 = 1,$$

$$x_3 = (1, 0)^T, y_3 = -1,$$

$$x_4 = (0, 1)^T, y_4 = -1,$$

请模拟感知机的运行过程（按 x_1 x_2 x_3 x_4 的顺序），直至正确分类。

- 1 概念
 - 假设
- 2 分类器
 - 参数选择
 - 超平面
- 3 感知机算法
 - 感知机算法
 - 几何直觉
- 4 感知机的收敛性
 - 感知机的收敛性
 - 定理与证明
- 5 感知机的历史
 - 感知机的历史
 - 从感知机到人工神经元
- 6 几何直觉

感知机的收敛性

感知机是一个具有强收敛性保证的算法。即：如果一个数据集是线性可分的，感知机将在有限次更新中找到一个分离的超平面。
(如果数据不是线性可分的，它将永远循环)

分析如下：

假设 $\exists \mathbf{w}^*$ ，使得 $\forall (\mathbf{x}_i, y_i) \in D, y_i(\mathbf{x}^\top \mathbf{w}^*) > 0$

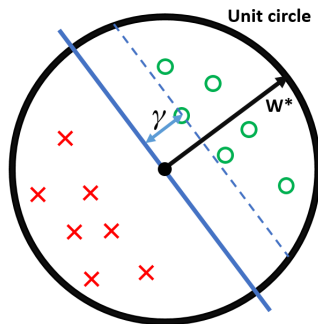
为方便分析，我们重新缩放每个数据点和 $\mathbf{w}^*$ ，使得

$$\|\mathbf{w}^*\| = 1 \quad \text{且} \quad \forall \mathbf{x}_i \in D, \|\mathbf{x}_i\| \leq 1$$

注：通过对每个数据点 $\mathbf{x}_i$ 进行缩放来完成，即均除以： $\alpha = \max_j \|\mathbf{x}_j\|$

感知机的收敛性

我们定义超平面 w^* 的 Margin γ 为: $\gamma = \min_{(x_i, y_i) \in D} |x_i^\top w^*|$.



总结一下我们的设置:

- 所有输入 x_i 位于单位球内
- 存在一个由 w^* 定义的分超平面, $\|w\|^* = 1$ (即 w^* 恰位于单位球上)。
- γ 是这个超平面 (蓝色) 到最近数据点的距离

定理: 若以上假设都成立, 则感知机算法最多会出现 $\frac{1}{\gamma^2}$ 次错误。

证明:

基于上述定义, 考虑更新 (w 更新为 $w + yx$) 对 $w^\top w^*$ 和 $w^\top w$ 两项的影响。

我们将利用以下两个事实:

- $y(x^\top w) \leq 0$: 这是因为 x 被 w 错误分类了一否则我们不会进行更新。
- $y(x^\top w^*) > 0$: 这是因为 w^* 是一个分离超平面, 其正确分类了所有的点。

1. 考虑 $\mathbf{w}^\top \mathbf{w}^* \Rightarrow (\mathbf{w} + y\mathbf{x})^\top \mathbf{w}^*$ 的影响:

$$(\mathbf{w} + y\mathbf{x})^\top \mathbf{w}^* = \mathbf{w}^\top \mathbf{w}^* + y(\mathbf{x}^\top \mathbf{w}^*) \geq \mathbf{w}^\top \mathbf{w}^* + \gamma$$

这是因为: 对于 $\mathbf{w}^*$, $\mathbf{w}^*$ 定义的超平面到 $\mathbf{x}$ 的距离必须至少为 γ (即 $y(\mathbf{x}^\top \mathbf{w}^*) = |\mathbf{x}^\top \mathbf{w}^*| \geq \gamma$)。

这意味着对于每一次更新, $\mathbf{w}^\top \mathbf{w}^*$ 至少增加 γ .

2. 考虑 $\mathbf{w}^\top \mathbf{w} \Rightarrow (\mathbf{w} + y\mathbf{x})^\top (\mathbf{w} + y\mathbf{x})$ 的影响:

$$(\mathbf{w} + y\mathbf{x})^\top (\mathbf{w} + y\mathbf{x}) = \mathbf{w}^\top \mathbf{w} + \underbrace{2y(\mathbf{w}^\top \mathbf{x})}_{<0} + \underbrace{y^2(\mathbf{x}^\top \mathbf{x})}_{0 \leq \leq 1} \leq \mathbf{w}^\top \mathbf{w} + 1$$

该不等式来自如下分析:

- $2y(\mathbf{w}^\top \mathbf{x}) < 0$: 当我们进行了一次更新之后, 意味着 $\mathbf{x}$ 被错误分类了
- $0 \leq y^2(\mathbf{x}^\top \mathbf{x}) \leq 1$, 因为 $y^2 = 1$ 且都有 $\mathbf{x}^\top \mathbf{x} \leq 1$ (because $\|\mathbf{x}\| \leq 1$).

这意味着对于每一次更新, $\mathbf{w}^\top \mathbf{w}$ 的增长幅度至多为 1.

3. 现在我们可以把上面的推导放在一起。假设我们做了 M 次更新：

$$M\gamma \leq \mathbf{w}^\top \mathbf{w}^* \quad \text{By first point} \quad (1)$$

$$= |\mathbf{w}^\top \mathbf{w}^*| \quad \text{Simply because } M\gamma \geq 0 \quad (2)$$

$$\leq \|\mathbf{w}\| \|\mathbf{w}^*\| \quad \text{By Cauchy-Schwartz inequality}^* \quad (3)$$

$$= \|\mathbf{w}\| \quad \text{As } \|\mathbf{w}^*\| = 1 \quad (4)$$

$$= \sqrt{\mathbf{w}^\top \mathbf{w}} \quad \text{by definition of } \|\mathbf{w}\| \quad (5)$$

$$\leq \sqrt{M} \quad \text{By second point} \quad (6)$$

$$\quad \quad \quad (7)$$

$$\Rightarrow M\gamma \leq \sqrt{M} \quad (8)$$

$$\Rightarrow M^2\gamma^2 \leq M \quad (9)$$

$$\Rightarrow M \leq \frac{1}{\gamma^2} \quad (10)$$

因此，更新的总次数 M 限界于一个常数。

* 替代解释： $|\mathbf{w}^\top \mathbf{w}^*| = \|\mathbf{w}\| \|\mathbf{w}^*\| |\cos(\alpha)|$, but $|\cos(\alpha)| \leq 1$

提问

基于上述定理，

- 1) 关于分类器的边界距离，边界距离大还是小更理想？
- 2) 感知器算法快速收敛的数据集具有什么特征？请试举一例。

提问

为方便分析，我们是通过对每个数据点 $\mathbf{x}_i$ 进行缩放来完成，即均除以： $\alpha = \max_j \|\mathbf{x}_j\|$
如果不做缩放，则最多迭代次数为： $M \leq ?$

提问

基于上述定理，

- 1) 关于分类器的边界距离，边界距离大还是小更理想？
- 2) 感知器算法快速收敛的数据集具有什么特征？请试举一例。

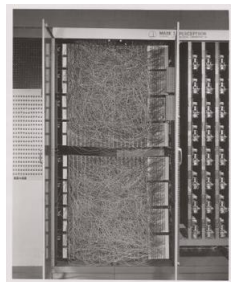
提问

为方便分析，我们是通过对每个数据点 x_i 进行缩放来完成，即均除以： $\alpha = \max_j ||x_j||$
如果不做缩放，则最多迭代次数为： $M \leq (\frac{\alpha}{\gamma})^2$

- 1 概念
 - 假设
- 2 分类器
 - 参数选择
 - 超平面
- 3 感知机算法
 - 感知机算法
 - 几何直觉
- 4 感知机的收敛性
 - 感知机的收敛性
 - 定理与证明
- 5 感知机的历史
 - 感知机的历史
 - 从感知机到人工神经元
- 6 几何直觉

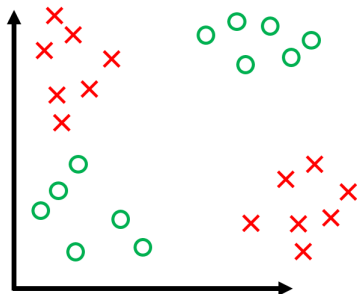
感知机的历史

- 感知机是 1957 年由康奈尔大学航空实验室的 Frank Rosenblatt 发明的。
- Mark I 感知机，是首个感知机算法的实现。
它连接到一个带有 20×20 硫化镉光电池的相机，可以拍摄 400 像素的图像。主要可见的特征是一个配线架，用于设置输入特征的不同组合。右边是实现自适应权重的电位器阵列。



感知机的历史

- 起初，引起了巨大的轰动（“数字大脑”）（见 1958 年 12 月的《纽约客》）
- 然后，终结于简单非线性可分离数据集的著名例子，异或问题（Minsky 1969）。导致了人工智能的冬天。



AND, OR, NOT, XOR

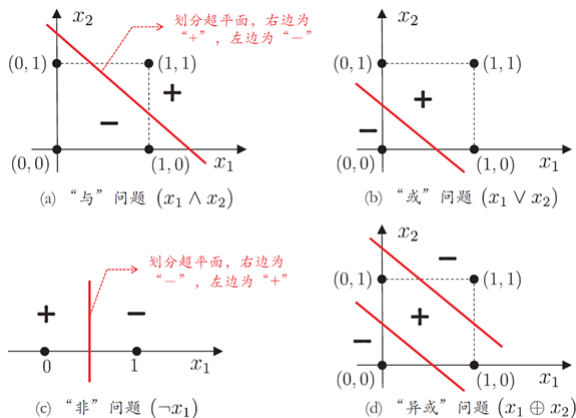
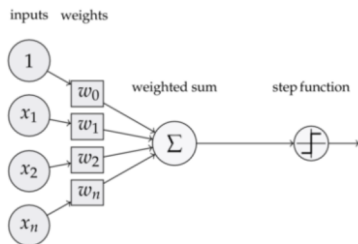


图 5.4 线性可分的“与”“或”“非”问题与非线性可分的“异或”问题

以神经元的方式理解感知机

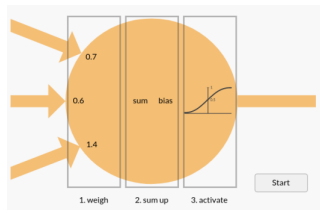
“Mark 1 感知机”是为图像识别设计的机器：它有 400 个光电管阵列，随机连接到“神经元”。权重被编码在电位器中，学习过程中的权重更新由电动机执行。



- 合法的分类超平面可能会有多个，甚至无穷个
- 感知机的解依赖于初值的选择、迭代过程中误分类点的顺序
- 如何得到一个最好的（同时可能也是唯一的）超平面？ $\Rightarrow$ 线性支持向量机
- 感知机存在对偶形式，支持向量机也存在对偶形式

从感知机到神经网络

- **Input:** 所有的特征都成为感知器的输入, $x = [x_1, x_2 \dots, x_n]$ 。
- **Weights:** 权重是在模型训练过程中计算的值。初始化时, 我们从某初始权重出发, 基于每次训练误差进行权重更新。 $w = [w_1, w_2, \dots w_n]$ 。
- **BIAS:** 偏置神经元使得分类器可以将决策边界向左或向右移动。用代数的术语, 偏置神经元允许分类器平移其决策边界。BIAS 有助于更快训练模型, 获得更好的性能。
- **加权求和:** 加权求和是将每个特征值与对应权重相乘后得到的值之和。
- **激活函数:** 激活函数的作用是使神经网络具有非线性。
- **输出:** 加权求和被传递给激活函数, 计算后得到的值即我们的预测输出。



- 1 概念
 - 假设
- 2 分类器
 - 参数选择
 - 超平面
- 3 感知机算法
 - 感知机算法
 - 几何直觉
- 4 感知机的收敛性
 - 感知机的收敛性
 - 定理与证明
- 5 感知机的历史
 - 感知机的历史
 - 从感知机到人工神经元
- 6 几何直觉

测试

1. 假设数据集仅由单个数据点 $\{(x, +1)\}$ 组成。感知器对这个点 x 反复错误分类的频率是多少？

设 $w = 0, m = 0$ 而 $y(w \cdot x) = 0 \leq 0$ 在经过一次迭代之后 $w = \{x, 1\}$
在第二次迭代时, $y(w \cdot x) = x^T \cdot x \geq 0$, 因此在经过一次分类错误后, 感知机就不会对点 x 反复错误分类。

2. 如果初始权重向量 w 是随机初始化的, 而不是全零向量会怎样?

假设不为全 0, 那么如果感知机的分类问题是可以二分类的, 那么一定可以收敛, 但是迭代次数会发生一定的变化。

测试

3. 对如下训练数据:

$$\mathbf{x}_1 = (3, 3, 1)^T, y_1 = 1,$$

$$\mathbf{x}_2 = (4, 3, 1)^T, y_2 = 1,$$

$$\mathbf{x}_3 = (1, 1, 1)^T, y_3 = -1,$$

初始时刻 $\mathbf{w} = 0$

第一轮迭代

对于 $\mathbf{x}_1, y_1 \cdot (\mathbf{w} \cdot \mathbf{x}_1) = 0 \leq 0, \mathbf{w} = 0 + (3, 3, 1)^T = (3, 3, 1)^T$

对于 $\mathbf{x}_3, y_3 \cdot (\mathbf{w} \cdot \mathbf{x}_3) = -7 \leq 0, \mathbf{w} = (3, 3, 1)^T - (1, 1, 1)^T = (2, 2, 0)$

对于 $\mathbf{x}_3, y_3 \cdot (\mathbf{w} \cdot \mathbf{x}_3) = -4 \leq 0, \mathbf{w} = (2, 2, 0)^T - (1, 1, 1)^T = (1, 1, -1)^T$

对于 $\mathbf{x}_3, y_3 \cdot (\mathbf{w} \cdot \mathbf{x}_3) = -1 \leq 0, \mathbf{w} = (1, 1, -1)^T - (1, 1, 1)^T = (0, 0, -2)^T$

对于 $\mathbf{x}_2, y_2 \cdot (\mathbf{w} \cdot \mathbf{x}_2) = -2 \leq 0, \mathbf{w} = (0, 0, -2)^T + (4, 3, 1)^T = (4, 3, -1)^T$

对于 $\mathbf{x}_3, y_3 \cdot (\mathbf{w} \cdot \mathbf{x}_3) = -6 \leq 0, \mathbf{w} = (4, 3, -1)^T - (1, 1, 1)^T = (3, 2, -2)^T$

对于 $\mathbf{x}_3, y_3 \cdot (\mathbf{w} \cdot \mathbf{x}_3) = -3 \leq 0, \mathbf{w} = (3, 2, -2)^T - (1, 1, 1)^T = (2, 1, -3)^T$

对于 $\mathbf{x}_3, y_3 \cdot (\mathbf{w} \cdot \mathbf{x}_3) = 0 \leq 0, \mathbf{w} = (2, 1, -3)^T - (1, 1, 1)^T = (1, 0, -4)^T$

对于 $\mathbf{x}_1, y_1 \cdot (\mathbf{w} \cdot \mathbf{x}_1) = -1 \leq 0, \mathbf{w} = (1, 0, -4)^T + (3, 3, 1)^T = (4, 3, -3)^T$

对于 $\mathbf{x}_3, y_3 \cdot (\mathbf{w} \cdot \mathbf{x}_3) = -4 \leq 0, \mathbf{w} = (4, 3, -3)^T - (1, 1, 1)^T = (3, 2, -4)^T$

对于 $\mathbf{x}_3, y_3 \cdot (\mathbf{w} \cdot \mathbf{x}_3) = -1 \leq 0, \mathbf{w} = (3, 2, -4)^T - (1, 1, 1)^T = (2, 1, -5)^T$

测试

检验

对于 $x_1, y_1 \cdot (\mathbf{w} \cdot \mathbf{x}_1) = 4 \geq 0$

对于 $x_2, y_2 \cdot (\mathbf{w} \cdot \mathbf{x}_1) = 6 \geq 0$

对于 $x_3, y_3 \cdot (\mathbf{w} \cdot \mathbf{x}_1) = 2 \geq 0$

感知机： 知错就改

《左传·宣公二年》：“过而能改，善莫大焉”

The End